Constructing an optimal batting order is a fundamental and recurring challenge in baseball, faced by managers at every level of the sport. Traditionally, lineup construction has been guided by a mixture of statistical reasoning and longstanding heuristics (for example, placing high on-base percentage hitters at the top of the order and power hitters in the middle). However, a lineup determines not only the sequence in which players bat, but also how often high-impact hitters come to the plate and how effectively offensive opportunities are converted into runs. These conventions, while intuitive, often fail to fully leverage the depth and granularity of modern player performance data. With the advent of Major League Baseball's Statcast system, teams now have access to highly detailed, pitch-by-pitch data capturing nuanced aspects of player performance, such as exit velocity, launch angle, and expected outcomes of batted balls. This creates an opportunity to explore whether machine learning methods can provide a more systematic, data-driven approach to lineup construction, potentially uncovering patterns not immediately apparent through conventional analysis.

In this project, we develop a two-model machine learning framework to study and predict batting order decisions. The first model operates at the player level, using offensive performance metrics to predict a probability distribution over the nine possible lineup positions. The second model builds on this representation by considering groups of nine players simultaneously, learning to predict a complete lineup configuration. By leveraging learned player embeddings and modeling interactions across players within a lineup, this approach aims to better approximate the joint decision-making process underlying real-world lineup construction. This framework extends prior work in operations research that models baseball as a probabilistic system, where lineup construction can be analyzed as an optimization problem, and player

sequencing influences run production.[1] While these approaches have provided valuable insights, they typically rely on fixed probabilistic assumptions.[2] In contrast, our method uses data-driven representations learned directly from high-dimensional Statcast features, allowing for greater flexibility in capturing complex, nonlinear relationships between player performance and lineup placement.

Our analysis is driven by two primary research questions. First, how accurately can we predict a player's lineup position based solely on their performance metrics up to a given point in the season? Second, which player attributes are most influential in determining lineup placement? More broadly, we seek to understand the extent to which lineup decisions in professional baseball can be explained and potentially replicated through statistical learning methods. The motivation for this work stems from both a personal passion for baseball and the richness of its data. From a machine learning perspective, the problem presents an interesting combination of classification and structured prediction tasks, while the sport itself still relies heavily on human judgment in strategic decisions. By applying modern learning techniques to lineup construction, we aim to bridge the gap between data availability and decision-making, providing insight into how quantitative methods can complement or challenge traditional approaches to the game.

The data for this project was sourced using the Python package pybaseball, which provides programmatic access to several major baseball databases, including Baseball Savant, Baseball Reference, and FanGraphs.[3] Specifically, we used the statcast function to retrieve

[1] Bukiet, Bruce, et al. "A Markov Chain Approach to Baseball." Operations Research, vol. 45, no. 1, Feb. 1997, pp. 14–23. pubsonline.informs.org (Atypon), https://doi.org/10.1287/opre.45.1.14.
[2] "Stochastic Modeling and Optimization in Baseball." Wiley Encyclopedia of Operations Research and Management Science, by James J. Cochran et al., 2011. DOI.org (Crossref), https://doi.org/10.1002/9780470400531.eorms0836.
[3] https://github.com/jldbc/pybaseball/tree/master/docs

pitch-by-pitch data for the entire 2025 Major League Baseball season. The raw Statcast dataset contains 118 variables per pitch, including pitch characteristics (location, velocity, spin rate), game context, player identifiers, and batted ball outcomes. To ensure consistency and representativeness of player performance, we restricted the dataset to players with at least 100 plate appearances during the season. We also standardized player identifiers using the playerid_reverse_lookup function, allowing us to consistently match players across datasets.[4] In addition to Statcast data, we incorporated a lineup dataset containing the batting order for each team in each game, including game date and home/away team designations. These two sources were merged using shared identifiers such as game date and team.

The unit of analysis in our final dataset is a player-game instance, where each row represents a single player's performance entering a specific game, along with their observed lineup position in that game. To construct meaningful input variables, we transformed pitch-level data into cumulative player statistics calculated up to each game date. These features were designed to capture multiple dimensions of offensive performance, including traditional rate statistics such as batting average (AVG), slugging percentage (SLG), and on-base percentage (OBP), and more advanced metrics like expected weighted on-base average (xwOBA). These variables were selected to provide a balanced representation of hitting ability, plate discipline, and quality of contact.

To prevent data leakage and better reflect real-world decision-making, all features were computed as cumulative statistics up to, but not including, the current game. This was implemented using cumulative sums with a one-step shift. Additional data cleaning steps

[4] Tools such as the Chadwick Register (which can also be accessed through PyBaseball) offer easy ways to choose between ID methods. The most up-to-date version is MLBID, but we choose to use Retrosheet which is more commonly used for historical data since that was the key type used by our game log of every lineup.

included removing rows with missing lineup positions (e.g., pinch hitters), ensuring there was always only one observation per player per game, and filtering out each player's first 10 games of the season to allow performance metrics to stabilize. The final dataset consists of 36,665 player-game observations across the 2,430 games of the 2025 MLB season. Each observation includes 11 engineered performance features, contextual variables such as game date, team, and player ID, and the target variable representing lineup position (1–9).